Code-pro Plugin - Full Specification

An exportable, editable Claude Code plugin that makes the assistant code like a professional senior full-stack developer across the whole development lifecycle. Its flagship skill, develop-fr, spends Claude Opus only on architecture and delegates implementation, test-writing, QA, and per-step review to Codex and Gemini.

How to use this document: drop this file into any capable AI assistant with the instruction "build this plugin exactly as specified". Everything needed is here: the file structure, all 9 skills, all 7 subagents, all 10 commands, the 4 helper scripts, the delegation contract, and build instructions. The two things you may want to edit between uses are Section 3 (Model & Effort) and Section 4 (Lanes).

Version 2.1 - lane backups. The delegation pipeline of 2.0 is unchanged; 2.1 adds awareness of quota exhaustion, the one failure mode the 2.0 degradation ladder did not cover. Preflight now reports any lane running on a backup implementer, and the optional delegate-backup plugin performs and reverses those swaps.

1. Plugin overview

Name: code-pro

Author: Somar

Version: 2.1.0

Purpose: Help the user code like a professional senior full-stack developer; support everyday tasks across all development life cycles: investigation, small and full development, bug fixing, review, testing, regression, refactoring, and documentation.

Target platform: Claude Code (.claude-plugin/plugin.json + skills/ + agents/ + commands/ + scripts/).

Distribution: One git repository serving as a Claude Code marketplace - github.com/SomarRezq/pro-marketplace. Install with /plugin marketplace add SomarRezq/pro-marketplace.

External dependencies (optional but strongly recommended): the delegate-skills collection (github.com/amElnagdy/delegate-skills) plus the codex and agy CLIs. Optional soft dependency: the delegate-backup plugin (github.com/SomarRezq/pro-marketplace/tree/main/claude/delegate-backup), which keeps the pipeline running when an implementer exhausts its quota. Without it nothing changes; with it, preflight gains a lane-backups report.

2. Coding philosophy (binding for every skill)

These seven principles define how the user wants AI to code. Every skill and agent below embeds them; any AI building or extending this plugin must preserve them.

- Match the repo, don't impose - structure, naming, UI style, and test conventions always come from the existing codebase, never from the AI's preferences.

- Smallest change that fully works - simplicity, SOLID principles, no unrequested refactoring, fewest changes possible.
- Tests mirror the codebase - if similar code parts have unit tests, new code gets unit tests in the same framework and style.
- Always end with a report - what was done and how, what was tested, what still needs manual testing from the user (with step-by-step instructions).
- Never leave behind - obvious bug sources, exception sources, memory leaks, security holes, or structurally incorrect code.
- Understand before touching - investigate related code paths first; visualize with workflow charts when useful.
- Never trust a self-report - an executor claiming its tests passed is a claim, not evidence. Re-run the gates yourself before believing anything. (New in 2.0, and the principle that makes delegation safe.)

3. MODEL & EFFORT CONFIGURATION - EDIT HERE

Single source of truth for every subagent's model and effort. When switching model generations, edit ONLY this table and mirror the two model/effort lines in each agent file. Models: haiku / sonnet / opus / inherit (inherit = the main conversation's model). Effort: low / medium / high.

Note what changed in 2.0: developer, qa-engineer and code-reviewer are no longer the default workers for develop-fr. That work is delegated to external models; these three remain as the degraded fallback path used when no external implementer is available. The reasoning budget is concentrated in solution-architect.

Agent	Role	Model	Effort	Limits
solution-architect	Planning & final architecture review (develop-fr, large refactor). The reasoning budget.	opus	high	-
investigator	Read-only deep code investigation (investigate)	sonnet	high	read-only
developer	FALLBACK implementer for one planned step (develop-fr)	sonnet	high	-
qa-engineer	FALLBACK end-to-end QA pass (develop-fr)	sonnet	high	-
code-reviewer	Review (code-review); FALLBACK per-step reviewer (develop-fr)	sonnet	high	read-only
regression-tester	Impact analysis + regression runs (regression-test)	sonnet	high	-
doc-writer	Writes docs from verified findings, token-efficient (create-docs)	haiku	high	no code edits

Skills WITHOUT an agent run inline with whatever model the user selected for the session - by design: develop, bug-fix, test, and small refactors.

4. LANE CONFIGURATION - EDIT HERE

A lane binds a kind of work to an external implementer CLI plus optional dials. Lanes are the delegation dial: they live in the user's delegate-skills fleet config, NOT in this plugin, so retuning the whole pipeline is one file and the plugin never hardcodes a vendor.

Config paths: ~/.config/delegate-skills/config.json (global) and <git-root>/.delegate/config.json (project, overrides by whole-lane replace).

Lane	Used for	Suggested configuration
feature	backend / logic implementation steps	{ "implementer": "agy", "model": "gemini-3.1-pro-high" }
ui	UI implementation steps	{ "implementer": "codex" }
tests	test-writing steps	{ "implementer": "codex" }
review	per-step independent code review	{ "implementer": "codex", "readOnly": true, "effort": "high" }
qa	end-to-end QA pass	{ "implementer": "codex" }
docs	documentation steps	{ "implementer": "agy", "model": "gemini-3.7-flash-high" }

Gemini reaches this pipeline through agy (the Google Antigravity CLI). There is no gemini-delegate skill - delegate-skills ships no other Google implementer - and the standalone gemini CLI is not used.

If a lane is missing from the fleet config, the plugin substitutes an all-Codex built-in default and says so in preflight. /code-pro-doctor prints the exact JSON to add.

5. Plugin file structure

```
claude/code-pro/
|-- .claude-plugin/plugin.json      # manifest: name, description, version, author
|-- README.md                       # skill map, pipeline diagram, model & lane tables
|-- skills/
|   |-- investigate/SKILL.md        #   each skill: YAML frontmatter (name, description)
|   |-- develop/SKILL.md            #   + Markdown body (goal, workflow, output format,
|   |-- develop-fr/                 #   example, guardrails)
|   |   |-- SKILL.md                #   the orchestrated pipeline (Section 6)
|   |   \-- references/
|   |       |-- orchestrator-contract.md   # the 'you MUST NOT' list
|   |       |-- brief-format.md            # the brief + Digest contract
|   |       |-- run-state.md               # run dir, state.json, checkpoints
|   |       \-- lanes-and-fallbacks.md     # lanes, capabilities, degradation
|   |-- bug-fix/SKILL.md
|   |-- code-review/SKILL.md
|   |-- test/SKILL.md
|   |-- regression-test/SKILL.md
|   |-- refactor/SKILL.md
|   \-- create-docs/SKILL.md
|-- agents/                         # 7 files, frontmatter: name, description, model,
|   \-- <agent-name>.md             #   effort, optional disallowedTools + system prompt
|-- scripts/                        # Node helpers, built-ins only, no dependencies
|   |-- lib.mjs                     #   lane resolution, discovery, capabilities
|   |-- preflight.mjs               #   detect CLIs / relays / lanes; report degradations
|   |-- run-init.mjs                #   create run dir, seed state.json, gitignore
```

```
|   |-- dispatch.mjs               #   lane -> implementer -> relay; normalize to Digest
|   \-- state.mjs                  #   run state, scheduling, checkpoints, resume
\-- commands/                      # 10 thin wrappers passing $ARGUMENTS
    \-- <command-name>.md
```

plugin.json content:

```
{
  "name": "code-pro",
  "description": "Code like a professional senior full-stack developer. Full lifecycle:
   investigate, develop, develop-fr, bug-fix, code-review, test, regression-test,
   refactor, create-docs. develop-fr reserves Claude Opus for architecture and delegates
   implementation, testing, and per-step review to Codex and Gemini.",
  "version": "2.0.0",
  "author": { "name": "Somar" }
}
```

6. The develop-fr pipeline

This section is the heart of version 2.0. It replaces the all-Claude pipeline of version 1.x.

6.1 Roles and where each model is spent

```
you --> ORCHESTRATOR (Claude - routing only, never reads or writes code)
   |
   |   every hop: write a brief file --> executor --> read a result Digest
   |
   |-0- PREFLIGHT ....... script, no model
   |-1- PLAN ............ Claude Opus / high      <- reasoning
   |-2- APPROVAL ........ the user
   |-3- PER STEP (parallel where dependencies allow):
   |      3a IMPLEMENT .. lane -> Gemini 3.1 Pro (agy) or Codex
   |      3b REVIEW ..... Codex, read-only
   |      3c REWORK ..... same session, <=2 rounds, then escalate
   |      3d GATES ...... the orchestrator re-runs them itself
   |-4- QA .............. lane -> Codex
   |-5- FINAL REVIEW .... Claude Opus / high      <- reasoning
   \-6- REPORT
```

Claude is touched at exactly three points: the plan, the final review, and the orchestrator's own short routing turns. Everything else is external.

The orchestrator is a pure router. references/orchestrator-contract.md states the prohibitions explicitly: it must not read source files, write or edit code, read a full diff, read a full executor report into context, accept a self-report as proof, decide parallelism by hand, invent gate commands, or expand scope. Those rules are the mechanism, not a style preference.

6.2 The delegation contract: one brief format, two transports

Every hop is the same shape - including the Claude-to-Claude ones. A brief is a self-contained Markdown file; the executor sees it and nothing else: no chat history, no repo memory, no knowledge of the other steps. If a fact is needed to do the work, it is in the brief or the work is wrong.

Brief sections: Goal, Context, Do, Do NOT, Conventions (quoted from real code, never described), Gates (the project's REAL commands), Definition of done, Report contract.

Transport A - an external model, via the delegate-skills relay:

```
node <plugin>/scripts/dispatch.mjs --brief <run>/steps/step-03.brief.md \
     --lane feature --cd <repo> --result <run>/steps/step-03.result.md --timeout 2h
```

Transport B - a Claude module, via a subagent given the same two paths:

```
Agent(subagent_type: "code-pro:solution-architect",
      prompt: "Your brief is at <abs>. Write your result to <abs>.
               Reply with the Digest only.")
```

Both write a result file whose first section is a Digest of at most 30 lines. The orchestrator reads only the Digest; full detail stays on disk for the next executor. This is what keeps Claude usage flat: a 40-step feature costs the orchestrator roughly what a 4-step feature costs.

```
## Digest
verdict: done | needs-decision | needs-changes | blocked
lane: feature
implementer: agy (gemini-3.1-pro-high)
status: completed - exit 0 - 214s
session: conv-7f2a91
files: src/routes/health.ts, src/app.ts
gates: npm run lint -> pass, npm test -> pass
open: none
```

dispatch.mjs builds the Digest from the relay's result.json, preferring the executor's own stated verdict over an inferred one, so a needs-decision is never silently downgraded to done.

Rework continues the SAME external session (--session for Codex, --conversation for Antigravity), so the implementer keeps its context and only the delta brief is paid for. Cap at 2 rounds; a third failure means the plan step is wrong, not the code, and escalates to the solution-architect.

6.3 Run state, checkpoints, and resuming

Everything the pipeline produces lives on disk inside the target repo, auto-gitignored on first run. Nothing important lives only in the orchestrator's context - which is what makes /compact lossless.

```
<repo>/.code-pro/runs/<YYYYMMDD-HHMMSS>-<slug>/
  00-request.md              the user's feature request, verbatim
  01-preflight.json          detected CLIs, resolved lanes, degradations applied
  02-architect.brief.md / .result.md      the plan, in prose
  plan.json                  machine-readable step index: id, title, deps[], lane
  steps/step-NN.brief.md / .result.md / .review.md / .rework-N.brief.md
  qa.brief.md / qa.result.md
  99-final-review.brief.md / .result.md
  state.json                 THE RESUME ANCHOR
  REPORT.md                  the final fixed-format output
```

plan.json drives parallelism. Steps whose deps are all done are dispatched together; state.mjs next is the scheduler and the orchestrator must not decide parallelism by reading the plan itself. Any two steps touching the same file MUST be dependency-linked - parallel writes to one file are the most likely way this pipeline corrupts a run. Import validates ids, unknown deps, self-deps and dependency cycles.

On the compact question: no skill or hook can force /compact at a threshold, so version 2.0 attacks the problem from the other side. The orchestrator's context barely grows (Digests only), and compaction is made lossless. The skill takes checkpoints after the plan, after every 3 completed

steps, and after QA, telling the user it is safe to compact. /develop-fr --resume recovers from state.json after a compaction, a crash, or in a brand-new session.

6.4 Preflight and graceful degradation

preflight.mjs runs first and records what it found. The pipeline always runs; it just reports honestly what it degraded to. Every degradation is printed once at the start and stored in 01-preflight.json - a silently-Claude run would defeat the purpose of the whole design.

Missing	Degrades to
delegate-skills not installed	in-Claude developer / qa-engineer / code-reviewer subagents
fleet config absent	the plugin's built-in all-Codex default lane map
the lane's implementer unusable	first usable of: codex, agy, copilot, opencode, cursor, claude
no external implementer at all	fully in-Claude, with a loud warning that the saving is lost

When an implementer is swapped, model and variant dials are dropped - a model label for one CLI is meaningless to another. dispatch.mjs exits 3 when a lane has no external implementer; that is the orchestrator's signal to spawn the in-Claude fallback agent for that role.

6.5 Implementer capabilities

Implementers differ in what they can do in the non-interactive mode the relays use, and one difference is significant enough to be encoded in dispatch.mjs.

Implementer	Message	Exhaustion? / window
codex	You have hit your usage limit	Yes - retry time
agy	Individual quota reached. Resets in XhYmZs	Yes - duration
opencode (Z.AI plan)	Weekly/Monthly Limit Exhausted. Your limit will reset at <timestamp>	Yes - absolute timestamp
copilot	You have exceeded your premium request allowance / HTTP 402 quota_exceeded	Yes - no window given
opencode (Z.AI)	Insufficient balance or no resource package (1113)	NO - configuration fault
opencode (Z.AI free)	Rate limit reached for requests	NO - throttle, retry

Antigravity cannot run gates. In print mode it denies every RunCommand, and if a brief tells it to run one, the run ends with no final message at all - the file edits land but the report is lost. Passing --sandbox does not change this. dispatch.mjs therefore appends an execution-environment note to briefs bound for a shell-less implementer, telling it not to run the gates and to report 'gates: not run (orchestrator verifies)'. The exact augmented text is written to <result>.effective-brief.md so nothing is hidden, and the Digest carries a matching warning.

This costs nothing, because phase 3d has the orchestrator re-run every gate itself regardless. dispatch.mjs --allow-shell opts into the implementer's own escape hatch (Antigravity: --dangerously-skip-permissions) - full access, so only on the user's explicit request.

6.6 Quota exhaustion and lane fallback chains (new in 2.1)

The degradation ladder in 6.4 answers one question: is this implementer present? It resolves usability as 'binary on PATH and relay installed', both evaluated once at preflight. That leaves a real gap - an implementer that is installed, authenticated, and simply out of quota is reported OK and then fails its dispatch.

Version 2.1 does not try to solve this inside the pipeline, because doing so well requires classifying provider errors, and provider errors are not trustworthy classifiers. The canonical counter-example: Z.AI returns 'Insufficient balance or no resource package' (error 1113) when the request used the wrong endpoint or a model outside the plan. It is a configuration fault wearing the words of an exhaustion fault. An automatic classifier would silently move work onto a backup provider to route around a bug the user needed to see.

So the split is: judgment stays with the orchestrator and the user; bookkeeping is automated by the optional delegate-backup plugin. Each lane gets an ordered CHAIN of implementers, and the plugin walks the lane one position down that chain per exhaustion, scheduling its return to the primary for when the provider's window resets. All of its state lives in a lane-backups.json sidecar, so delegate-skills' config.json remains a valid, untouched delegate-fleet.v1 document and stays free for any future delegate-skills upgrade.

What preflight does in 2.1:

- Reads lane-backups.json directly when it exists - a soft dependency, so a missing plugin costs one informational line rather than a failure.
- Prints a 'Lane backups (active)' block naming every lane on a fallback, its chain position as [n/last], what it fell back to, and how long until it returns. The position matters: it shows how much fallback headroom is left.
- Marks an entry OVERDUE when its restore window has passed, which means a scheduled restore never fired and 'delegate-backup resolve --all' should be run.

A lane on a fallback is being executed by a different provider than the fleet config names. Like every other degradation in this pipeline, that must never be silent. The chain's final entry should be a free, unmetered model - opencode/nemotron-3-ultra-free and opencode/hy3-free both write working

code at zero cost and need no credential - so that 'every provider is exhausted' degrades the quality of a run rather than stopping it.

Provider messages, and the three that mislead:

Implementer	Message on exhaustion	Reset window given?
codex	You've hit your usage limit (plus a retry time, or an admin request on Business seats)	Yes
agy	Individual quota reached. Resets in XhYmZs	Yes
copilot	You have exceeded your premium request allowance / HTTP 402 quota_exceeded	No
opencode (Z.AI)	Insufficient balance or no resource package (1113) - CONFIGURATION, not quota. Never swap on this.	n/a

Copilot reports no window, so a swap against it needs one chosen by the user. The others carry theirs in the error text and can be passed straight to delegate-backup's --until, which accepts both a duration and an absolute timestamp. Never advance a chain on 1113 or on a rate-limit message: the first is a wrong endpoint or an out-of-plan model, the second is a per-minute throttle that should simply be retried.

7. Skills (9)

Each skill's frontmatter description doubles as its trigger: it must state what the skill does AND when to use it, phrased slightly "pushy" so the AI doesn't under-trigger. The bodies explain the why behind rules, not just the rules.

Skill	Command	What it does	Agent(s)
investigate	/investigate	Visual workflow charts of existing code, read-only	investigator
develop	/develop	Small features/modifications, inline	-
develop-fr	/develop-fr	Full features via the delegating pipeline (Section 6)	solution-architect + external
bug-fix	/bug-fix	Root-cause fix, minimal change + regression test	-
code-review	/code-review	Severity-ranked read-only review + commit message	code-reviewer
test	/test	Unit tests mirroring repo test conventions	-
regression-test	/regression-test	Impact analysis + suite run, report-only	regression-tester

refactor	/refactor	Behavior-preserving restructure, approval required	solution-architect (large)
create-docs	/create-docs	Accurate docs with charts, investigated not assumed	doc-writer

Only develop-fr changed structurally in 2.0. The other eight keep their 1.x definitions: goal, numbered workflow, fixed output format, an example, and guardrails, each embedding the Section 2 philosophy.

7.1 develop-fr (the changed skill)

Trigger: the user wants a complete feature, brand-new functionality, or a massive change affecting a big part of the code or its structure.

Goal: a full feature delivered fully working, as simple as possible, in repo structure and style - while spending the strongest model only on architecture. If the repo has a constitution file (e.g. a spec-kit constitution), its implementation specs are binding.

Workflow

1. Input: a detailed explanation of the requested feature (ask targeted questions if too thin - a thin request produces a thin plan, and every downstream model inherits it).
2. PREFLIGHT - run-init.mjs creates the run directory; preflight.mjs records implementers, lanes and degradations. Tell the user once, plainly, what degraded.
3. PLAN - the solution-architect (opus, high) reads a brief file and writes the plan plus plan.json. It is the one participant allowed to read the codebase broadly.
4. APPROVAL - present the plan digest and get the user's approval. Then checkpoint: this is the largest single context drop available.
5. IMPLEMENT - for each wave of ready steps, write a self-contained brief and dispatch them in parallel to their lanes. Record implementer and session in state.json.
6. REVIEW - dispatch each finished step to the review lane (Codex, read-only) against that step's own definition of done. On needs-changes, send a delta brief on the same session; cap at 2 rounds.
7. GATES - the orchestrator re-runs the project's real gates itself. Only then is a step marked done. Checkpoint every 3 steps.
8. QA - dispatch the qa lane over the whole feature: success and failure paths, edge cases, and the seams between steps, which is where independently-implemented steps diverge.
9. FINAL REVIEW - the solution-architect (opus, high) reviews the full diff plus the QA digest for gaps, structural damage, and security or resource problems.
10. REPORT using the fixed output format, plus a one-line delegation summary showing which steps went to which model.

Output format (fixed)

```
## What was done and how
- <per step: what was built, where, key decisions, which model implemented it>
```

```
## What was tested
- <QA results, test suites run, outcomes>

## Needs manual testing
- <step-by-step manual verification instructions for the user>
```

Guardrails

- As simple as possible; repo structure, coding, naming and UI style everywhere.
- No refactoring of unrelated existing functionality.
- Constitution/spec file wins over any model's preferences.
- Never leave the result structurally incorrect, with bug sources, security holes, memory leaks, or untested parts.
- Three different models touch the codebase in one run. The plan's quoted conventions are the only thing keeping their output coherent - never let a brief ship without them.

8. Subagents (7)

One Markdown file per agent in agents/, YAML frontmatter carrying the model-effort setting marked for quick editing; keep in sync with Section 3. Below each: the agent's system-prompt essence - the builder AI should expand these into full prompts preserving every stated rule.

```
---
name: <agent-name>
description: <when the main AI should delegate to this agent>
model: sonnet            # <- MODEL-EFFORT (Section 3)
effort: high             # <-
disallowedTools: Write, Edit, NotebookEdit   # read-only agents only
---
<full system prompt in Markdown>
```

Read-only roles are enforced by removing the write tools, which is stronger than instructions alone - hence disallowedTools on investigator, code-reviewer and doc-writer.

8.1 solution-architect (rewritten in 2.0)

Two modes, both file-based: it reads a brief file and writes a result file, replying to the orchestrator with the Digest only.

Planning: studies the codebase and its conventions, honors any constitution/spec file, researches how such features are commonly built for this project type, settles UI/UX with the user when needed, then produces numbered steps - each with definition of done, how to test, constraints, structure decisions, directions, and dependencies. It also emits plan.json with a lane and deps[] per step. It must write for a stranger: every step is executed by a model with no memory of the repo, no chat history, and no sight of the other steps, so conventions must be quoted from real code rather than described.

Final review: given the full diff plus the QA digest, verifies everything requested is implemented, every definition of done is met, the seams between independently-implemented steps hold, the repo's conventions survived contact with three different models, and no loopholes, bug sources, memory leaks or security breach points remain. It reads the diff, not the whole repo.

8.2 The fallback trio (developer, qa-engineer, code-reviewer)

In develop-fr these three are the degraded path only, spawned when preflight finds no usable external implementer for a lane or when dispatch.mjs exits 3. They carry the same brief/result file contract as every other executor. code-reviewer additionally remains the primary agent for the standalone code-review skill.

developer - Executes one step of an approved plan - and only that step. Matches the repo exactly (structure, style, naming down to parameter names, error handling, UI style) by reading neighboring code first. SOLID, fewest changes meeting the definition of done. Writes and runs tests when the plan or repo conventions call for them. Self-verifies, reports files changed, test results and DoD verdict - and escalates uncovered decisions instead of guessing.

qa-engineer - Independent QA pass: runs relevant suites, exercises every planned use case and path (success AND failure), tries realistic edge cases, verifies each step's definition of done, and tests the feature end to end - the joints, not just the parts. Fixes nothing; reports pass/fail per case with exact reproduction steps, plus what needs the user's manual testing.

code-reviewer - Reviews in priority order: correctness, security, resource/memory handling, error handling, repo conventions & SOLID (learned from neighboring code), test coverage. Every finding carries file:line, the issue, why it matters as a concrete consequence, and a suggested fix; findings without consequences are dropped. Returns summary, Critical/Major/Minor findings, coverage gaps, verdict, and a suggested commit message.

8.3 The unchanged three

investigator - Read-only. Traces every entry point and code path of a feature (function to function, input -> output per step, success AND failure branches), marks UI/API/DB boundary crossings, and notes visible risks with file:line. Returns structured findings the orchestrator turns into charts; labels unconfirmed paths "unverified".

regression-tester - Maps the blast radius of a change (callers, shared state, events, DB touchpoints, config), runs the existing suite or the affected scope while stating which, and writes missing regression tests in repo style. Never fixes breakage and never modifies existing tests to make them pass.

doc-writer - Writes only what verified findings support - real behavior, never invented or intended behavior; ambiguities become open questions. Matches repo doc conventions, includes Mermaid flow charts for every documented workflow, and adapts depth to audience. Creates doc files only.

9. Commands (10)

Thin wrappers: frontmatter with a one-line description, body = "Use the <skill> skill on: $ARGUMENTS" plus a one-paragraph restatement of the skill's non-negotiables. They live in commands/ and are invoked as /name. One per skill, plus the new doctor.

Command	Non-negotiable reminder in the body

/investigate	Read-only; charts + notes
/develop	Inline, small scope, repo style
/develop-fr	Orchestrator routes and never writes code; read the orchestrator contract first
/bug-fix	Root cause, smallest change, regression test
/code-review	Read-only; severity-ranked; suggested commit message
/test	Mirror the repo's framework and style; tests must run and pass
/regression-test	Report-only; never fix breakage
/refactor	Behavior-preserving; plan approval required before touching code
/create-docs	Investigated from code, never assumed
/code-pro-doctor	Verify setup; do not change configuration without asking

10. Scripts (4 + shared lib)

Node helpers the skills shell out to. Node built-ins only - no dependencies, because this ships inside a plugin and must never need an install.

Script	Purpose	Key behavior
lib.mjs	Shared helpers	Lane resolution with the degradation ladder, implementer and relay discovery, cross-platform which, implementer capability table
preflight.mjs	Setup check	Detects CLIs, relays and lanes; writes 01-preflight.json; exit 1 means degraded
run-init.mjs	Start a run	Creates the run tree, seeds state.json, adds .code-pro/ to .gitignore idempotently
dispatch.mjs	Delegate one brief	lane -> implementer -> that implementer's relay; augments briefs for shell-less implementers; normalizes result.json into a Digest; exit 3 = use the Claude fallback
state.mjs	Run state	digest / next / step / phase / import-plan / checkpoint / latest; validates plan.json including dependency cycles

dispatch.mjs does not reimplement delegate-skills - it calls their relays. That is precisely why the pipeline is implementer-agnostic and why swapping a lane requires no plugin change.

11. Distribution

The plugin lives in a single public git repository that acts as a Claude Code marketplace.

```
pro-marketplace/
|-- .claude-plugin/marketplace.json   # marketplace manifest (must be at repo root)
|-- claude/code-pro/                  # the plugin (Section 5)
```

```
|-- docs/                            # this specification + the refactor plan
\-- README.md

/plugin marketplace add SomarRezq/pro-marketplace
/plugin install code-pro@pro-marketplace

# updating
/plugin marketplace update pro-marketplace
```

Bump the version in the plugin manifest and its marketplace entry whenever you ship changes.

A Codex port of this plugin existed under codex/ through version 1.1 and was removed in 2.0; it remains recoverable from the codex-port-v1 git tag. Note that this is unrelated to code-pro using the codex CLI as an implementer, which is a core part of the current design - do not let one be re-added by confusion with the other.

12. Build instructions for the AI reading this file

11. Create the folder structure from Section 5, including the plugin manifest exactly as shown.
12. Create one SKILL.md per skill in Section 7. Frontmatter: name + the trigger text as description (third person, includes when-to-use). Body: goal with the why, numbered workflow, fixed output format as a template, the example, and guardrails. Embed the Section 2 philosophy.
13. Build develop-fr from Section 6, with its four reference files. The orchestrator contract is not optional prose - it is the mechanism that makes the design work, and must be stated as prohibitions.
14. Create one agent file per Section 8, expanding the system-prompt essence into a full prompt. Take model and effort ONLY from Section 3; keep those two lines clearly commented for easy retuning. Mark the fallback trio as the degraded path in both their descriptions and their bodies.
15. Write the five scripts from Section 10 using Node built-ins only. Every failure mode must be loud: an unknown lane, a missing relay, an invalid plan.json, and every applied degradation.
16. Create the ten command files per Section 9.
17. Validate: every skill description states when to trigger; every agent has model + effort matching Section 3; read-only roles carry disallowedTools; every skill body ends in its fixed report format; all JSON parses; every script passes node --check.
18. Verify against reality, not appearance: run the doctor, dry-run a dispatch per lane, then run one real read-only and one real write dispatch through each configured implementer. Confirm the written file is correct AND that the gates pass when you run them yourself. A pipeline that looks right on paper and fails on first contact is the expected outcome without this step.
19. Package and distribute per Section 11.